# GS08 · Matrículas y Notas

- **Curso GS08 · Docente: Dr. Jack Harry Peralta Cáceres**
- Arteaga Rullier, Roberto — Scrum Master, DevOps y Backend
- Inocencio Gargate, Dalcir — Frontend y Documentación
- *Presentación grupal · 21/09/2026*

# El problema y el alcance del MVP

- Hoy: PHP + MySQL sin API, sin paginado, reglas dispersas en PHP, buscador LIKE que en PostgreSQL devuelve 0 filas sin avisar, y sin notas.
- Entra: autenticación, panel con KPIs, estudiantes, cursos, matrículas, usuarios, NOTAS y boleta, más contenedores, CI y Kubernetes.
- 39 criterios de aceptación con ID único (C-01…C-39): cada uno es un comando o una cifra que se puede comprobar.
- NO entra (20 puntos, por decisión): portal del estudiante, rol docente, pagos, horarios, 2FA, app móvil, nube, backups.

# Arquitectura y stack

- SPA React 19 + nginx → 2 instancias del API FastAPI → PostgreSQL 16, todo en contenedores sobre una red Docker.
- FastAPI 0.141.1 · PostgreSQL 16.15 · React 19 + Vite + Tailwind · Docker 29.8.0 · Prometheus v3.13.3 + Grafana 12.4.11.
- Cinco diagramas documentados y verificados: los 5 renderizan sin error.
- Decisión central: se reimplementa el legacy, no se migra — se conservan el modelo de datos y la UX, no el código.

# Modelo de datos y reglas de negocio

- 4 tablas + 1 vista traducidas de MySQL 8 a PostgreSQL 16: 16 CHECK, 6 UNIQUE, 2 FK y 7 índices.
- Las reglas RN-01…RN-16 viven en la base (última línea de defensa); la API las repite para responder 422 con mensaje.
- Semilla oficial: 12 estudiantes, 7 cursos, 24 matrículas (23 activas + 1 retirada) y 1 usuario admin.
- 54 comprobaciones del modelo y 32 de notas, con pruebas negativas que exigen que la base rechace cada regla violada: 0 fallas.

# API: contrato y comportamiento

- 20 rutas y 35 operaciones en su propio OpenAPI (v0.3.0), más GraphQL en /graphql: salud, autenticación, panel, estudiantes, cursos, matrículas, usuarios, notas, boleta, auditoría y boleta archivada.
- Códigos como contrato: 200/201/204 lo correcto · 400 invalid_grant o invalid_scope · 401 sin sesión · 403 por rol o por scope · 404 · 409 duplicado · 422 dato inválido.
- Ningún dato del usuario puede provocar un 500: el mensaje del backend se muestra en pantalla.
- Todas las operaciones se consumen desde el SPA y se verificaron contra la API real; los dos 500 de los POST de matrículas y usuarios quedaron cerrados.

# La SPA: el recorrido del usuario

- Entrar → panel con los 4 indicadores → buscar «huaman» y encontrar «Huamán» → registrar estudiante → matricular → 3 notas → ver la boleta → cursos, usuarios y la pantalla de Auditoría (bitácora y boleta archivada).
- Los errores se muestran con el mensaje del backend (409 y 422) y sin diálogos nativos del navegador.
- Menú por rol: un usuario asistente no ve ni puede llamar la pantalla de usuarios.
- Estado real: el SPA existe y consume las 30 operaciones del contrato; el recorrido se verificó de punta a punta en el navegador contra la API real.

# Migración del frontend: Vue 3 → React 19

- Por qué: la rúbrica del trabajo colaborativo de Implementación de Soluciones para Plataformas Web pide React con Tailwind. Se migra el frontend para cumplirla.
- Qué cambia: solo la capa de presentación. La SPA está desacoplada del API —todo pasa por /api/v1—, así que no se toca el backend, ni la base de datos, ni la infraestructura.
- Qué se gana: de 4 vistas al consumo de las 30 operaciones del contrato, con cursos, usuarios, notas y boleta, más búsqueda, filtros, paginación y pruebas.
- Despliegue: mismo build estático servido por nginx, misma imagen y el mismo Coolify. El cambio no altera el entorno publicado: se vuelve a construir y listo.

# Contenedores y CI

- 8 servicios con un solo comando: web (nginx: 80 en producción, 8080 en local), dos instancias del API, PostgreSQL, MongoDB, Redis, Prometheus y Grafana.
- Balanceo real 1 a 1 entre las dos instancias; con una apagada, 6 peticiones siguen dando 200 con reintento (failover).
- CI en cada push a main: infraestructura + backend (ruff y 121 pruebas, 90 % de cobertura) + frontend (25 pruebas, eslint y build) + imágenes con humo; los cinco jobs están en verde.
- CD en GitHub Actions: al terminar el CI en verde sobre main pide el despliegue a Coolify y verifica producción

# Capa de rúbrica del API: segunda base, caché, GraphQL y logging

- Segunda base documental (MongoDB 8.0): la bitácora guarda usuario, acción, entidad, resultado y duración de cada escritura, con contraseñas y tokens enmascarados («***»).
- Caché distribuida (Redis 8): el panel se arma una vez y se guarda 60 s; medido con 20 000 estudiantes y 20 000 matrículas: 54,2 ms → 13,5 ms (4,0×), y cualquier escritura la invalida sola.
- GraphQL en /graphql (Strawberry) con la consulta de boleta, que comparte una sola fórmula de promedio ponderado con el REST.
- Logging centralizado: JSON con identificador de petición, usuario, ruta, estado y duración; Grafana lee la serie {servicio="gs08-api"} desde Loki.

# Kubernetes local y observabilidad

- 11 objetos definidos y validados offline con kustomize: Namespace, ConfigMap, PVC, Job de migraciones, Deployments, Services e Ingress.
- En el cluster el esquema lo aplica Alembic con el Job de migraciones, no los scripts de inicialización.
- Observabilidad corriendo: 3 targets up y un dashboard API RED con 9 paneles con datos, provisionado desde el repositorio.
- **ESTADO REAL: el cluster NO está desplegado en esta máquina — minikube no está instalado y espera aprobación (A-02).**

# Calidad: 39 criterios y lo que apareció probando

- Los 39 criterios del alcance quedan cubiertos con corrida real: recorrido E2E de 39 OK y las pruebas automatizadas (121 del backend con 90 % de cobertura y 25 del SPA).
- 9 bugs del sistema y 4 del propio script de pruebas, todos cerrados con evidencia: un bug no se cierra por decisión, se cierra con la corrida verde pegada.
- Verificar en el navegador encontró 3 cosas que las pruebas simuladas no ven (D-31): un fallo transitorio de Mongo o Redis apagaba la bitácora para siempre, la pantalla de Auditoría decía «apagada» cuando la consulta había fallado, y una sesión vencida dejaba un estado falso.
- **Regla de la casa: un bug no se cierra por decisión, se cierra con la corrida verde pegada.**

# Seguridad, accesibilidad y producción

- OAuth2 con scopes: POST /api/v1/auth/token (password grant, RFC 6749 §4.3) con 16 scopes repartidos por rol; si al token le falta el permiso responde 403 insufficient_scope con WWW-Authenticate.
- Accesibilidad WCAG 2.1 AA medida: foco atrapado en los diálogos, «Saltar al contenido», aria-current en el menú, caption y scope en las tablas; 14/14 pares de contraste cumplen AA y el anillo de foco 5.93:1.
- 25 pruebas del SPA con axe-core y 121 del backend con 90 % de cobertura; los cinco jobs del CI en verde en cada push.
- **En producción (https://gs08-matriculas.mlarca.com): deploy verificado el 14/10 — SPA React sirviendo, login con 16 scopes, bitácora activa y 20/20 llamadas al health sin fallos.**

# Sprint 0 → Sprint 4: los cinco cerrados y la retrospectiva

- Sprint 0 a Sprint 4: los cinco cerrados con evidencia (alcance, modelo verificado, API, SPA, capa de rúbrica, producción y documentación).
- Sprint 1 y 2: API núcleo con contrato probado y el SPA con siete pantallas en React 19; los dos 500 de los POST cerrados.
- Sprint 3 y 4: notas y boleta con una sola fórmula de promedio, capa de rúbrica completa (MongoDB, Redis, GraphQL, OAuth2 con scopes, logging y accesibilidad) y publicación en producción.
- Único bloqueo declarado: Kubernetes real sin desplegar (A-02); Express.js queda como decisión abierta (A-04) con plan B de un día.